



南方科技大学

LAB 10: Model Quantization

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Introduction to Quantization	1
The LLM Quantization	9
Post-Training Quantization (PTQ)	11
Quantization for Fine-Tuning (PEFT)	17
The Quantization Ecosystem	24
Lab 10 Exercises	26

Introduction to Quantization

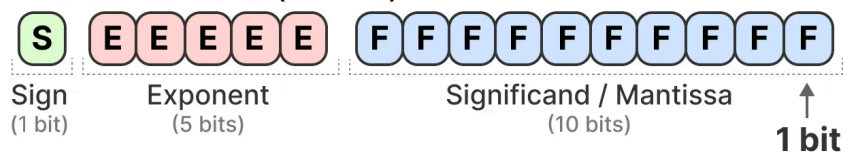
Fitting massive models onto consumer GPUs.

How Computers Store Numbers (FP16)

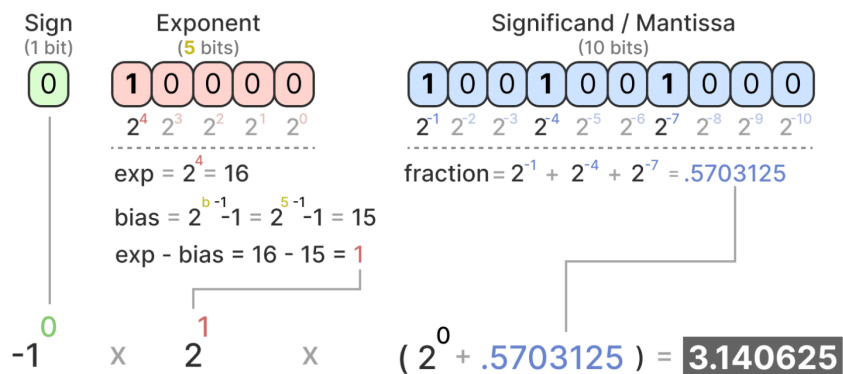


南方科技大学

Float 16-bit (FP16)



Float 16-bit (FP16)



1. Bits and Binary

Computers store numbers in **bits** (0 or 1).

N bits can represent 2^N unique values.

(e.g., 2 bits $\rightarrow$ 00, 01, 10, 11)

2. FP16 (Half-Precision, E5M10)

The default format for LLMs. **IEEE 754** standard:

$$\text{Value} = (-1)^S \times 1.M \times 2^{E - \text{bias}}$$

Divides the **16 bits** (2 bytes) into three parts (**E5M10**):

- **Sign (S , 1 bit):** + or -.
- **Exponent (E , 5 bits):** Determines the **Range**.
- **Mantissa (M , 10 bits):** Determines the **Precision**.

The Need for Quantization



南方科技大学

LLMs face a severe **Memory Capacity** issue: a 671B model in FP16 needs $671 \text{ billion param} \times 2 \text{ bytes/param} = 1.34\text{TB}$ just for weights!

Quantization: Mapping FP32/FP16 to lower-precision formats like INT8/INT4.

Data Compression:

- **FP16:** 16 bits (standard).
- **INT8:** 8 bits ($2\times$ smaller).
- **INT4:** 4 bits ($4\times$ smaller).

The Payoff (30B model):

FP16: **60 GB** $\rightarrow$ INT8: **30 GB** $\rightarrow$ INT4: **15 GB**

A consumer GPU (24GB) can now comfortably fit 30B!

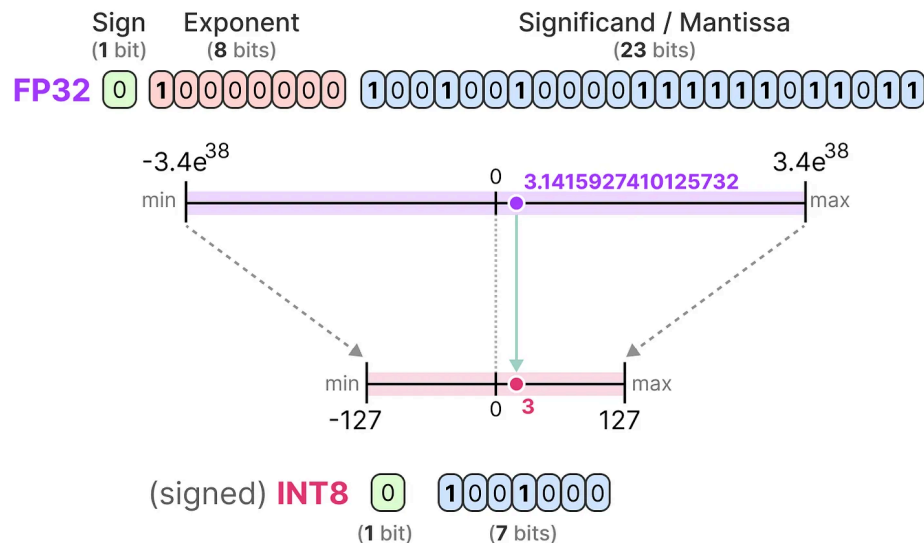
Beyond Capacity: Hardware Efficiency

- **Compute:** Modern GPUs have specialized circuits that execute lower-precision math much faster (e.g. Nvidia A100: FP32 = 19.5 TFLOPS, INT8 = 624 TFLOPS).
- **Memory:** Moving data (VRAM $\rightarrow$ SRAM) is $4\times$ faster, breaking the **Memory Wall** (Recap Lab 8).

FP32 to INT8: Direct Casting vs Quantization



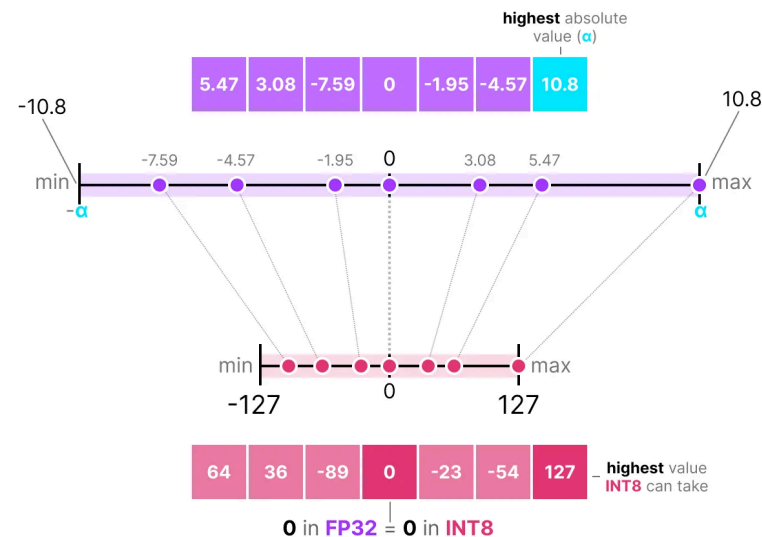
南方科技大学



Naive Direct Casting

Convert like `i = int(3.14159)` in Python.
Most weights collapse to 0 → Information lost!

Think: Why FP32 → BF16 direct casting is not an issue?



Quantization

Map the full floating-point range to integer bounds (e.g. $[-128, 127]$). → To preserve the information as much as possible.

Two Types of Quantization Error



南方科技大学

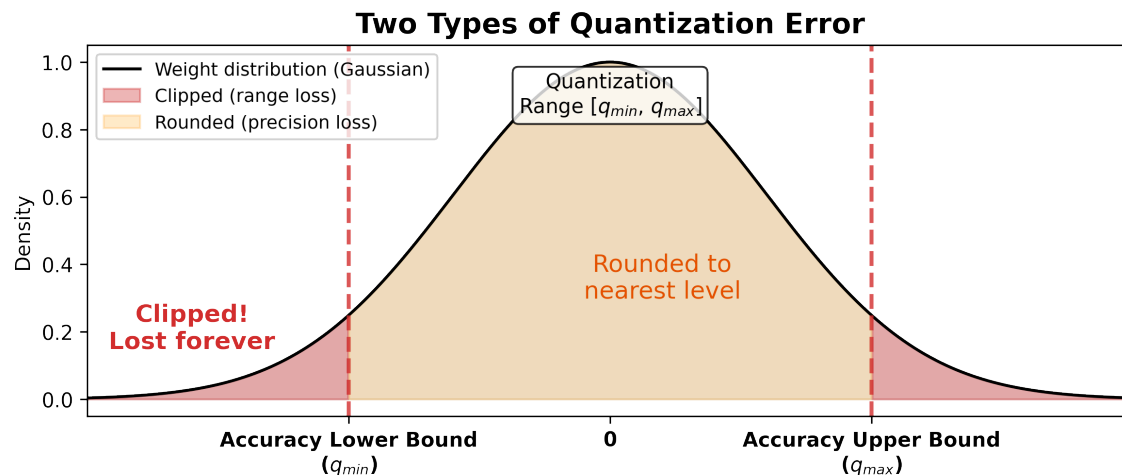
When we map FP32 $\rightarrow$ INT8, every value suffers one of two errors:

Range Loss (Clipping)

Values outside $[x_{\min}, x_{\max}]$ are **clipped**.

Precision Loss (Rounding)

Values inside range are **rounded**, e.g. 3.141592, 3.141593 $\rightarrow$ Same INT8 value.



Clipping vs. Rounding Error

LLM Tolerance to Errors are different:

- **Rounding:** Neural networks are highly robust to small, uniform noise (like rounding).
- **Clipping:** LLMs are **sensitive** to clipping outliers, which can destroy the model capability.

Affine Quantization (Linear Mapping)



南方科技大学

Map floating-point range $[x_{\min}, x_{\max}]$ to integer range $[q_{\min}, q_{\max}]$.

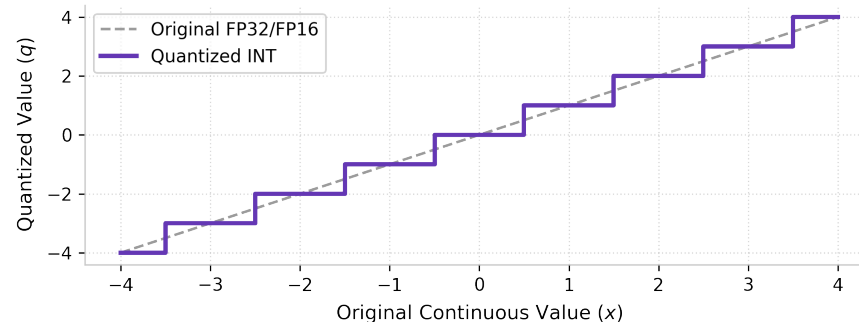
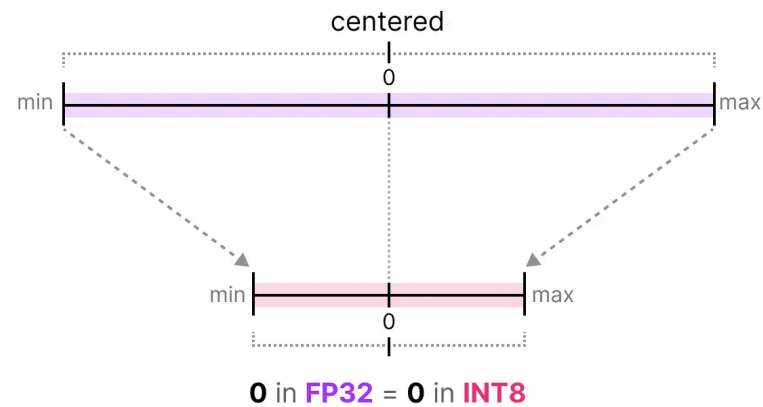
$$q = \text{clip}\left(\text{round}\left(\frac{x}{S}\right) + Z, q_{\min}, q_{\max}\right)$$

- x : Original float. q : Quantized int.
- S : **Scale factor** (step size).
- Z : **Zero-point** (maps float 0 to an integer).

Dequantization:

$$x_{\text{approx}} = S \times (q - Z)$$

Quantization Error = $x - x_{\text{approx}}$.



Continuous to Discrete Mapping

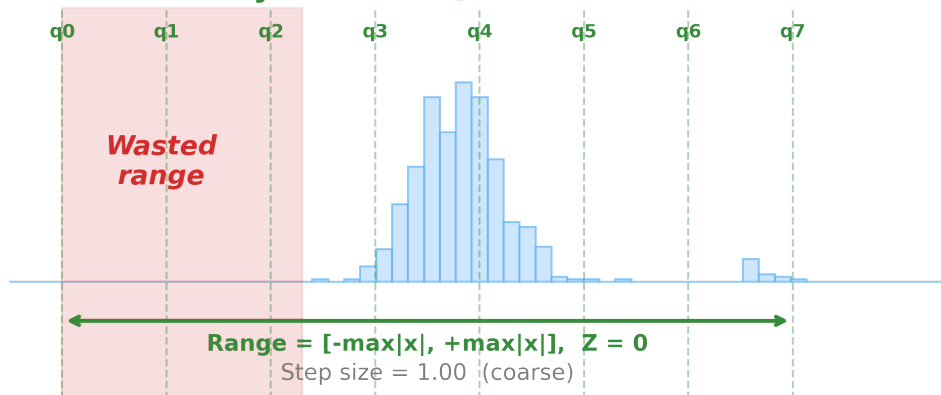
Symmetric vs Asymmetric Quantization



Symmetric (Center = 0)

- Range: $[-\max(|x|), \max(|x|)]$.
- Zero-point $Z = 0$ (forced).
- $S = \frac{\max(|x|)}{q_{\max}}$
- **Faster**: no zero-point math.

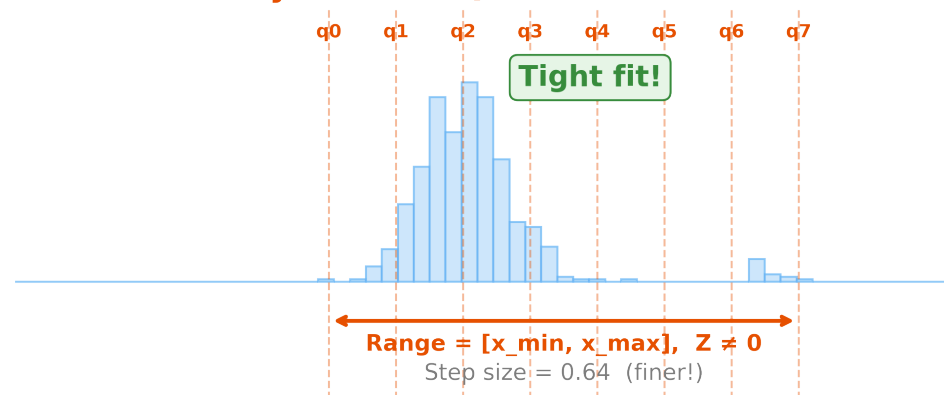
Symmetric Quantization



Asymmetric (Center $\neq 0$)

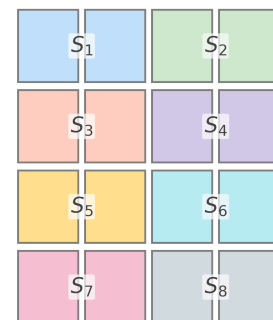
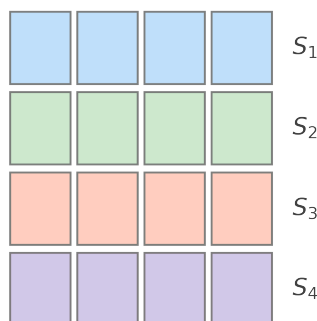
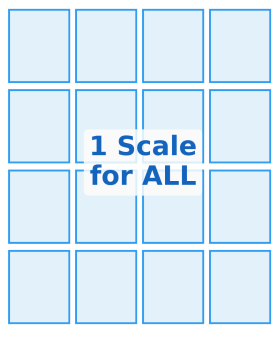
- Range: $[x_{\min}, x_{\max}]$.
- $S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$, $Z = q_{\min} - \text{round}\left(\frac{x_{\min}}{S}\right)$
- Better use of bits to reduce rounding error.
- **Better for skewed distributions** (e.g., ReLU).

Asymmetric Quantization



Symmetric wastes range on skewed data; Asymmetric fits tightly, needs extra zero-point calculation.

A single scale S for an entire matrix? Extreme values ruin the scale for all values.



Per-Tensor

One S , Z for entire matrix. Efficient but poor accuracy.

Per-Channel

One S , Z per output channel. Standard for weight quantization.

Group-wise

One S , Z per block of N elements. E.g., $N = 64, 128$.

Finer granularity = better accuracy, more metadata overhead.

The LLM Quantization

How to maintain the LLM accuracy under low precision.

Depending on the application scenario, quantization splits into two paradigms:

Post-Training Quantization (PTQ) 🔑

- **Goal:** High-speed Inference.
- **Method:** Quantize a pre-trained model offline before deployment.
- **Ecosystem:** AWQ, GPTQ, GGUF.

Quantization for Fine-Tuning 🔧

- **Goal:** Low-memory Training (PEFT).
- **Method:** Freeze base weights in 4-bit, train small FP16 adapters (LoRA).
- **Ecosystem:** bitsandbytes, QLoRA.

What about QAT (Quantization-Aware Training)?

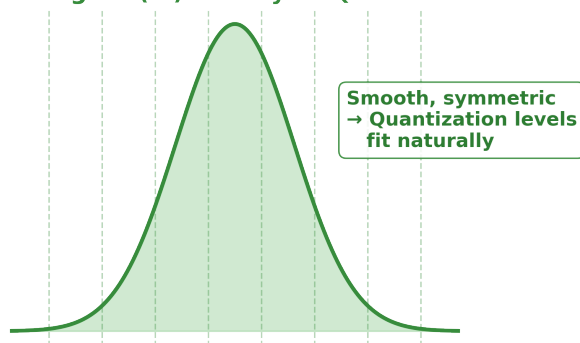
Training models from scratch in low-precision (e.g. 1-bit or INT4) is an active research area (like BitNet), but remains extremely difficult and costly to stabilize for massive LLMs.

Post-Training Quantization (PTQ)

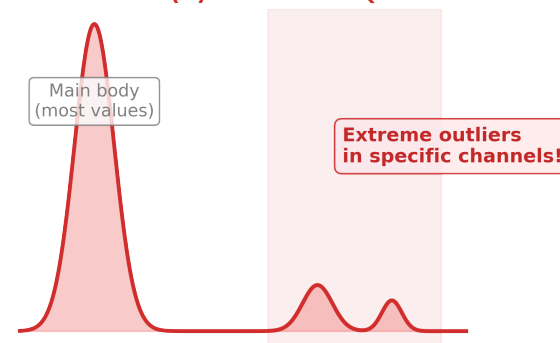
How to protect model intelligence during compression.

LLM Basic Operation (Linear Layer): $Y = WX$

Weights (W) — Easy to Quantize



Activations (X) — Hard to Quantize



Model Weights W

- Static, fixed after training.
- Smooth bell-curve distribution (Gaussian).
- **Relatively easy** to quantize (INT4, INT8).

Activations X (intermediate)

- Dynamic, changes every input.
- Extreme outliers in specific channels.
- **Extremely hard** to quantize.

The PTQ Challenge: Activation Outliers



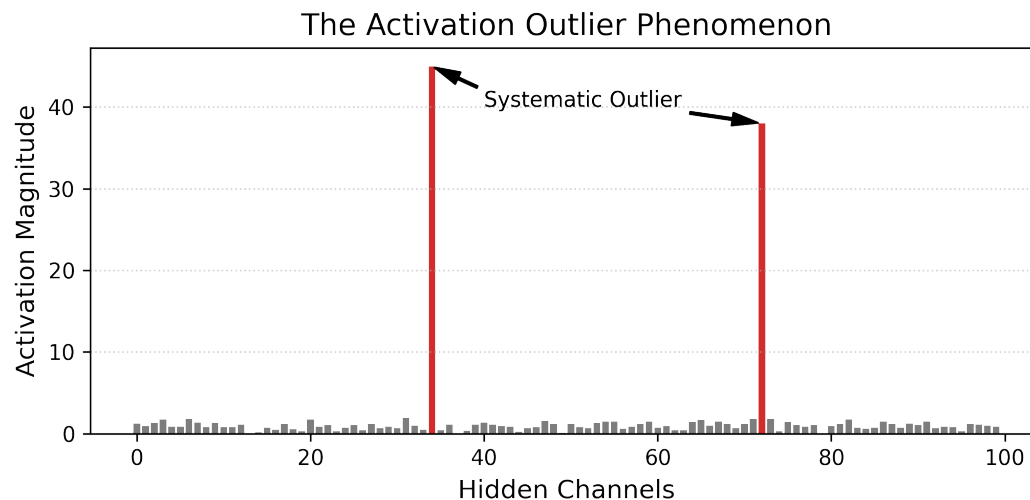
南方科技大学

Why not just quantize all the weights into low precision (e.g., 4-bit)?

The Outlier Phenomenon

At larger scales (Billion-level params), LLM activations develop **massive systematic outliers** in specific hidden dimensions.

With Per-Tensor quantization, one outlier forces S to be huge $\rightarrow$ normal values get crushed to zero $\rightarrow$ **model destroyed!**



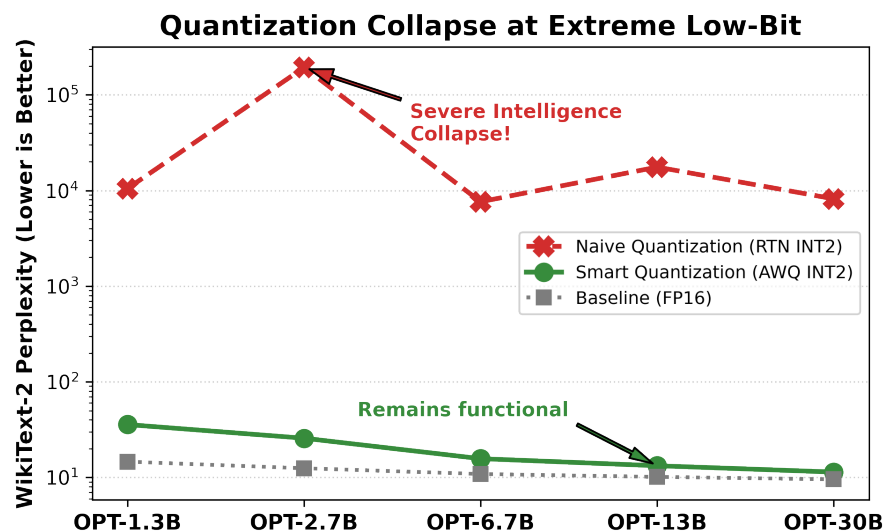
Activation magnitudes across channels, note the outliers in red boxes

The Quantization Tradeoff



南方科技大学

Quantization is a **Tradeoff** between Hardware Cost (Memory/Speed) and Accuracy.



OPT model with INT2 quantization (AWQ Paper)

Naive Quantization (RTN)

Treats all weights equally.

At extreme low-bit (e.g., 2-bit), quantization error destroys the model Intelligence.

Smart Quantization (AWQ/GPTQ)

Protects the 1% important weights.

Result: Massive memory savings while maintaining near-baseline accuracy.

The Insight

Only 1% of weights are “important” (produce huge activation magnitudes).

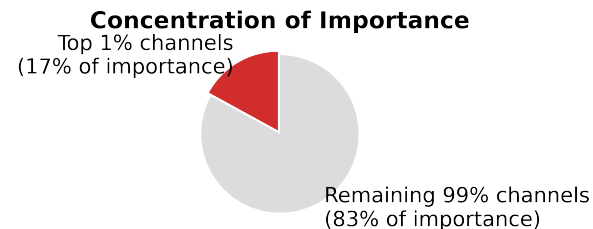
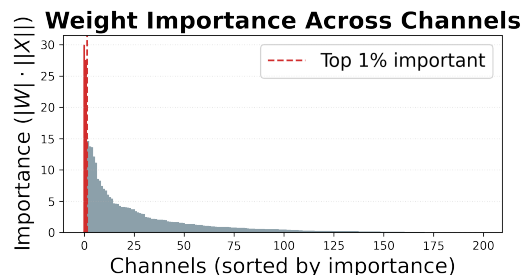
W4A16 Quantization

- **A16**: Keep all **A**ctivations in 16-bit to preserve dynamic outliers.
- **W4**: Identify the 1% important **W**eights, but **quantize 100% of weights to 4-bit**.

How to identify the 1%? (Activation-Aware)

For layer $Y = WX$, if X has **outliers**, W in this layer must be protected.

Small Quant. Error $\times$ **Outliers** = **Big Output Error**



Not all weights are equal: 1% share 17% importance

🤔 **Why not keep the 1% important weights in FP16 (Mixed Precision W)?**

Constraint: GPUs cannot do multiply between FP16 and INT8.

Question: How do we mathematically protect this 1% while forcing ALL weights into INT4 storage?

AWQ: Protection of 1% Important Weights



南方科技大学

All weights must be INT4, yet the 1% must stay accurate. **Solution: The Scaling Trick.**

$$Y = WX \quad \Rightarrow \quad Y = \underbrace{(W \cdot s)}_{\text{scale up}} \cdot \underbrace{\left(\frac{X}{s}\right)}_{\text{scale down}}$$

Example: INT4 step $\Delta = 0.5$, weight value $w = 0.12$

Without Scaling

$w = 0.12$, nearest bin: $Q(0.12) = 0.0$

Reconstructed: $\hat{w} = 0.0$

Relative Error = $|0.12 - 0| / 0.12 = \mathbf{100\%}$

With Scaling ($s = 5$)

$w' = 0.6$, nearest bin: $Q(0.6) = 0.5$

Dequantize: $0.5/5 = 0.1$

Relative Error = $|0.12 - 0.1| / 0.12 = \mathbf{16.7\%}$

Key: Scaling up $W \rightarrow$ salient weights occupy more INT4 bins $\rightarrow$ lower relative error.

Since W are normally distributed, scaling rarely causes clipping.

Quantization for Fine-Tuning (PEFT)

How to train massive models on limited hardware.

From Lab 9: LoRA freezes the base model and trains tiny bypass matrices.

Full Fine-Tuning

Updates every weight W .

Huge optimizer states (Adam: $12 \times$ model size
= optimizer + weight + gradients + activation).

LoRA (Low-Rank Adaptation)

Freeze W , inject trainable A , B (rank r).

$$W_{\text{new}} = W + BA.$$

Only $r \cdot (d + k)$ params trained.

But: LoRA saves **optimizer** memory. You still need to load the base model W in FP16!
A 7B model still costs 14 GB just to sit in VRAM. How do we shrink W itself?

Mixed-precision strategy: **Quantized storage + High-precision updates.**

$$\text{Training Objective: } Y = \underbrace{\text{deq}(W_{\text{NF4}})}_{\text{Frozen Base}} X + \underbrace{BA}_{\text{Trainable LoRA}} X$$

Base Model W

- Stored: **4-bit NF4**
- Frozen: No updates
- Saves 75% memory

Adapters A, B

- Stored: **16-bit**
- Trainable params
- High-precision opt.

Computation

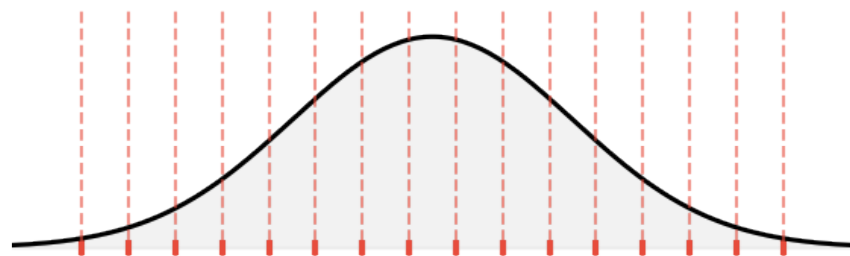
- Format: **BF16**
- On-the-fly dequant
- Prevents overflow

Rationale: LoRA parameters are tiny ($<0.1\%$) $\rightarrow$ high-precision updates are cheap. NF4 ensures the base model remains near-lossless.

Hardware: SFT 7B on consumer GPUs (16GB), 65B on 48GB. Accessible SFT for everyone!

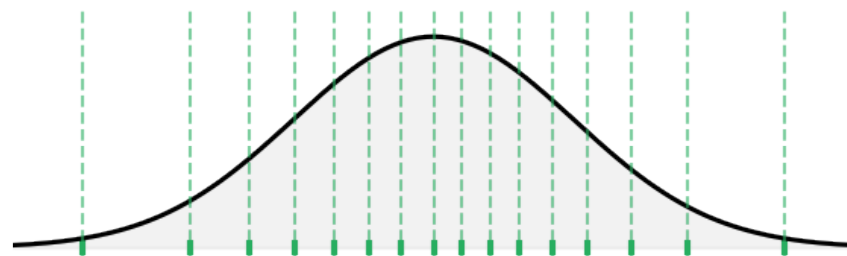
Why does QLoRA use **NF4** over standard INT4? W supposed to be Gaussian-distributed.

Standard INT4



Standard INT4 Waste More Precision in the Tail

NormalFloat4



NormalFloat4 Optimize Precision in the Center

Optimal Quantization: To minimize **total rounding error**, allocate more precision to the most frequent values. NF4 achieves this by ensuring each bin has an **equal probability** (Max Entropy).

1. **Scale:** Normalize input tensor to $[-1, 1]$ by dividing by $|W|_{\max}$.
2. **Quantize:** Map each element to the closest bin in the NF4 Lookup Table.

✓ **Result:** More parameters stay high-precision with minimal rounding error.

QLoRA in Practice: Loading the 4-bit Base Model



南方科技大学

The `bitsandbytes` library provides the backend for QLoRA model quantization.

```
1  import torch
2  from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
3  from peft import prepare_model_for_kbit_training
4
5  # 1. Configure Quantization Parameters (The core of QLoRA)
6  bnb_config = BitsAndBytesConfig(
7      load_in_4bit=True,                # Enable 4-bit quantization
8      bnb_4bit_quant_type="nf4",        # Use NF4 (NormalFloat4) data type
9      bnb_4bit_use_double_quant=True,    # Double Quantize: quantize the scales too!
10     bnb_4bit_compute_dtype=torch.bfloat16 # Dequantize to BF16 for matmul (prevents overflow)
11 )
12
13 # 2. Load the Quantized Base Model & Tokenizer
14 model_id = "meta-llama/Meta-Llama-3-8B"
15 tokenizer = AutoTokenizer.from_pretrained(model_id)
16
```

Python

QLoRA in Practice: Loading the 4-bit Base Model



南方科技大学

```
17 model = AutoModelForCausalLM.from_pretrained(  
18     model_id,  
19     quantization_config=bnb_config,  
20     device_map="auto"                # Auto dispatch to available GPUs  
21 )  
22  
23 # 3. Prepare for Low-Precision Training  
24 # Enables gradient checkpointing & freezes all 4-bit base layers  
25 model = prepare_model_for_kbit_training(model)
```

QLoRA in Practice: LoRA Adapters & Training



南方科技大学

Use the `peft` and `trl` libraries to inject trainable adapters and fine-tune.

```
1  from peft import LoraConfig, get_peft_model
2  from transformers import TrainingArguments
3
4  # 4. Configure and Inject the LoRA Adapters, same to Lab 9
5  lora_config = LoraConfig(
6      r=16,                                # LoRA rank (size of the adapters)
7      lora_alpha=32,                        # Scaling factor (usually 2x the rank)
8      target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], # Attach to Attention layers
9      task_type="CAUSAL_LM"
10 )
11 model = get_peft_model(model, lora_config) # Inject LoRA into the 4-bit model
12 model.print_trainable_parameters()
13 ... # Train the LoRA adapters
```

 Python

The Quantization Ecosystem

Navigating the diverse landscape of tools.

The Hugging Face Quantization Ecosystem



南方科技大学

Quantization Method	On the fly quantization	CPU	CUDA GPU	ROCm GPU	Metal (Apple Silicon)	Intel GPU	Torch compile()	Bits	PEFT Fine Tuning	Serializable with 🤖 Transformers	Transformers Support
AQLM	🔴	🟢	🟢	🔴	🔴	🟢	🟢	1/2	🟢	🟢	🟢
AutoRound	🔴	🟢	🟢	🔴	🔴	🟢	🔴	2/3/4/8	🔴	🟢	🟢
AWQ	🔴	🟢	🟢	🟢	🔴	🟢	?	4	🟢	🟢	🟢
bitsandbytes	🟢	🟢	🟢	🟡	🟡	🟢	🟢	4/8	🟢	🟢	🟢
compressed-tensors	🔴	🟢	🟢	🟢	🔴	🔴	🔴	1/8	🟢	🟢	🟢
EETQ	🟢	🔴	🟢	🔴	🔴	🔴	?	8	🟢	🟢	🟢
Four Over Six	🟢	🟢	🟢	🔴	🔴	🔴	🟢	4	🔴	🟢	🟢
FP-Quant	🟢	🔴	🟢	🔴	🔴	🔴	🟢	4	🔴	🟢	🟢
GGUF / GGML (llama.cpp)	🟢	🟢	🟢	🔴	🟢	🟢	🔴	1/8	🔴	See Notes	See Notes
GPT-QModel	🔴	🟢	🟢	🟢	🟢	🟢	🔴	2/3/4/8	🟢	🟢	🟢

Source: Hugging Face Ecosystem

<https://huggingface.co/docs/transformers/quantization>

There is no “one size fits all” quantization method. Hugging Face integrates many backends to cover all needs.

Key Trade-offs to Consider:

- **Hardware Support:** Different backends support different quantization methods (e.g. NVIDIA: AWQ/GPTQ, CPU: GGUF).
- **Bit Width:** 8-bit, 4-bit (standard) are common.
- **Application:** Inference v.s. PEFT Fine Tuning.

Lab 10 Exercises

Bringing models down to size.

Task 1: Linear Quantization (`lab10_quantize.py`) [40 points]

- Implement affine quantization and dequantization functions.
- Measure the Mean Squared Error (MSE) between original and dequantized weights.

Task 2: Granularity Comparison (`lab10_quantize.py`) [30 points]

- Implement Per-Tensor and Per-Channel quantization.
- Compare their reconstruction errors on a skewed weight distribution.

Task 3: QLoRA Fine-Tuning (`lab10_qlora.py`) [30 points]

- Fine-tune a model using QLoRA on the exact same dataset used in Lab 9.
- Compare the VRAM and training speed against the standard FP16 LoRA baseline.

Task 4: Implementing NF4 Quantization (`lab10_nf4.py`) [Bonus]

NF4 is information-theoretically optimal for Gaussian weights.

1. **Derive** the bins. Write a script to find the 16 values that divide the $\mathcal{N}(0, 1)$ distribution into **equal-probability** regions.
2. **Mapping:** Implement a vectorized `quantize_nf4(tensor)` function that maps weights to these derived bins.
3. Compare the reconstruction MSE against standard INT4 on real LLM layers.

Hint: Use `torch.distributions.normal.Normal.ppf` or `scipy.stats.norm.ppf` to get the distribution's inverse CDF (Cumulative Distribution Function, 累积分布函数).

Task 4 Supplementary



南方科技大学

To validate your implementation, you can compare your derived bins :

```
[-1.0, -0.7076, -0.5422, -0.4168, -0.3109, -0.2159, -0.1273, -0.0421, 0.0421, 0.1273, 0.2159, 0.3109, 0.4168, 0.5422, 0.7076, 1.0]
```

Note: Your derived bins may differ slightly from the official `bitsandbytes` NF4. The official version ensures an exact 0.0 and uses a specific asymmetric quantization for better hardware efficiency. Our task focuses on the **theoretical Gaussian optimal**.

Happy Coding!